

TRABAJO INTEGRADOR MATEMÁTICA

Docentes: Federico Rodriguez
Eloy Molina

Integrantes: Delfina Negro
Carolina Carranza
Renata Serafini
Angelina Ankovic

Consigna 1 — Validación de usuarios y análisis de consistencia del sistema

PARTE A — Análisis con conjuntos

1. Calcular e interpretar:

a) Usuarios que utilizan ambas plataformas:

$$A \cap B = \{104, 105, 106\}$$

b) Usuarios que utilizan al menos una plataforma:

$$A \cup B = \{101, 102, 103, 104, 105, 106, 107, 108\}$$

c) Usuarios que utilizan la plataforma, pero no presentan errores:

$$(A \cup B) - C = \{101, 103, 104, 106, 107, 108\}$$

d) Usuarios que utilizan exclusivamente una sola plataforma:

$$A \Delta B = \{101, 102, 103, 107, 108\}$$

2. Expresar al menos dos resultados usando comprensión de conjuntos:

$$\{x \in \mathbb{N} \mid x \in A \wedge x \in B\}$$

$$\{x \in \mathbb{N} \mid x \in A \vee x \in B\}$$

3. Detectar:

a) Usuarios que aparecen en C pero no en $A \cup B$:

$$C - (A \cup B) = \{109\}$$

Parte B — Lógica proposicional

5. Construir la tabla de verdad:

p	q	r	$p \vee q$	$(p \vee q) \wedge r$
V	V	V	V	V
V	V	F	V	F
V	F	V	V	V
V	F	F	V	F
F	V	V	V	V
F	V	F	V	F
F	F	V	F	F
F	F	F	F	F

6. Implementar una función en Python que evalúe la expresión:

```
A = [101, 102, 103, 104, 105, 106]
B = [104, 105, 106, 107, 108]
C = [102, 105, 109]

usuarios = [101, 102, 103, 104, 105, 106, 107, 108, 109]

print("Usuarios críticos:")

for usuario in usuarios:

    p = usuario in A
    q = usuario in B
    r = usuario in C

    if (p or q) and r:
        print(usuario)

print("Usuarios no críticos:")

for usuario in usuarios:

    p = usuario in A
    q = usuario in B
    r = usuario in C

    if not((p or q) and r):
        print(usuario)
```

7. Clasificar los usuarios del dataset en:

- Críticos
- No críticos

Usuarios	$(p \vee q) \wedge r$	Clasificación de Usuarios
101	F	No Crítico
102	V	Crítico
103	F	No Crítico
104	F	No Crítico
105	V	Crítico
106	F	No Crítico

Usuarios	$(p \vee q) \wedge r$	Clasificación de Usuarios
101	F	No Crítico
102	V	Crítico
103	F	No Crítico
107	F	No Crítico
108	F	No Crítico
109	F	No Crítico

Parte C — Análisis

8. Responder:

- ¿Qué tipo de usuario representa mayor riesgo?

Los usuarios críticos son los que representan un mayor riesgo ya que pertenecen a $C(r)$, es decir, los usuarios que han generado errores (ID 's: 102-105). Esto puede afectar el funcionamiento del sistema, provocar fallos y generar problemas para otros usuarios.

- ¿Qué significa que un usuario esté en C pero no en $A \cup B$?

Significa que el usuario presenta errores registrados, pero no aparece utilizando ninguna de las plataformas. En este caso, es el ID: 109.

- ¿Qué decisión tomarían como equipo programador?

Como equipo programador, nosotros prestaríamos más atención a los usuarios críticos, porque son los que cumplen la condición de riesgo.

Podríamos controlar mejor sus acciones, revisar su actividad y agregar medidas de seguridad para evitar problemas en el sistema.

CONSIGNA 2 - Análisis y optimización de costos de desarrollo

PARTE A - Análisis matemático

1. $A(x)$: Pendiente = 40
Ordenada al origen = 200
 $B(x)$: Pendiente = 70
Ordenada al origen = 50
2. Las funciones A y B NO son paralelas ya que tienen pendientes diferentes.

3. **Coordenada X:**

$$\begin{aligned}40x + 200 &= 70x + 50 \\40x - 70x &= 50 - 200 \\-30x &= -150 \\x &= -150 / -30 \\x &= 5\end{aligned}$$

Coordenada Y:

$$\begin{aligned}Y &= 40x + 200 \\y &= 40 * 5 + 200 \\y &= 200 + 200 \\y &= 400\end{aligned}$$

El punto de intersección de las funciones A y B se encuentra en las coordenadas $x=5$, $y=400$

4. **FUNCIÓN $C(x) = -2x^2 + 80x + 100$**

$$\begin{aligned}a &= -2 \\b &= 80 \\c &= 100\end{aligned}$$

Vértice:

$$\begin{aligned}X_v &= \frac{-b}{2a} \\X_v &= \frac{-80}{2 * (-2)} \\X_v &= \frac{-80}{-4} \gg X_v = 20\end{aligned}$$

$$\begin{aligned}Y_v &= -2 * (20)^2 + 80 * (20) + 100 \\Y_v &= -2 * (400) + 80 * (20) + 100 \\Y_v &= -800 + 1600 + 100 \gg Y_v = 900\end{aligned}$$

$$X_v = 20, Y_v = 900$$

Raíces:

$$x = -b \pm \frac{\sqrt{b^2 - 4ac}}{2a}$$

$$x = -80 \pm \frac{\sqrt{80^2 - 4*(-2)*100}}{2*(-2)}$$

$$x = -80 \pm \frac{\sqrt{6400 - (-800)}}{-4}$$

$$x_1 = -80 + \frac{\sqrt{7200}}{-4} \gg x_1 = -1.21$$

$$x_2 = -80 - \frac{\sqrt{7200}}{-4} \gg x_2 = 41.21$$

PARTE B - Implementación

5. Se realizó un programa que calcula y muestra datos sobre funciones lineales o cuadráticas. Inicia con un menú que permite al usuario elegir entre los dos tipos de funciones para luego permitir ingresar los valores correspondientes a cada función. Los datos evaluados de cada función son:

FUNCIÓN LINEAL:

- **Pendiente:** Es directamente el valor **m** ingresado por el usuario.
- **Ordenada al origen:** Es directamente el valor **b** ingresado por el usuario.
- **Raíz:** Se usó la fórmula de raíz para función lineal: $-b / m$
 - Si $m \neq 0$: Se usa la fórmula sin problemas.
 - Si $m = 0$ y $b \neq 0$: La función es una recta horizontal que flota arriba o abajo del eje X, por lo tanto, el programa le avisa al usuario que **no tiene raíces** (nunca toca el eje).
 - Si $m = 0$ y $b = 0$: La recta está apoyada justo sobre el eje X, por lo que el programa informa que existen **infinitas raíces**.
- **Comportamiento:** Se analiza si la función es creciente, decreciente o constante teniendo en cuenta el valor de la pendiente:
 - Si m es positiva la función es creciente.
 - Si m es negativa la función es decreciente
 - Si m es 0 la función es constante.
- **Ángulo:** Para precisión y organización se utilizó la función de python “math” que permite conocer el arcotangente de la pendiente m directamente y pasar el valor que originalmente da en radianes, a grados.
- **Dominio:** En todas las funciones lineales el dominio es el conjunto de todos los números reales (-infinito, +infinito).
- **Rango:** Se analiza según el comportamiento de la pendiente:
 - Si la función es **creciente o decreciente** ($m \neq 0$): el rango abarca todos los números reales (-infinito, +infinito) porque la recta se proyecta infinitamente hacia arriba y hacia abajo.
 - Si la función es **constante** ($m = 0$), la recta es completamente horizontal y el rango se reduce únicamente al valor de la ordenada al origen (b), ya que la función nunca cambia de altura.

FUNCIÓN CUADRÁTICA:

- **Ordenada al origen:** Es directamente el valor **c** ingresado por el usuario.
- **Vértice:** Se utiliza la fórmula de vértice en x: $-b / (2 * a)$
- **Concavidad:** Se analiza el signo del coeficiente a:
 - Si $a > 0$, la parábola es cóncava hacia arriba.

- Si $a < 0$, la parábola es cóncava hacia abajo.
- **Raíces:** Se calculan evaluando el discriminante.
 - Si es mayor a 0: El programa calcula las **dos raíces reales** distintas, las calcula y las muestra.
 - Si es igual a 0: Informa que tiene **una sola raíz**, el vértice que toca el eje X, la calcula y la muestra.
 - Si es menor a 0: Informa que **no tiene raíces reales** (gráficamente no toca el eje X, da raíces imaginarias).
- **Dominio:** Al igual que la lineal, matemáticamente no tiene restricciones a menos que se aplique en un problema real, que no es el caso de este programa, por lo que abarca todos los números reales (-infinito, +infinito)
- **Rango:** Depende de la concavidad y de la altura del vértice (v_y).
 - Si es cóncava hacia arriba: El rango va desde el vértice al infinito positivo (v_y , +infinito),
 - Si es cóncava hacia abajo: El rango viene del infinito negativo hasta la altura del vértice (-infinito, v_y).

SEGUNDO MENÚ INTERACTIVO:

Se inicia con un bucle `while` $\neq 3$ para que el menú siga apareciendo mientras el usuario siga eligiendo otras opciones

1. **Generar gráfico de la función:** Para ello se implementaron librerías de python
2. **Obtener valor de Y ingresando valor de X:** Se pide valor de x y se reemplaza x de la ecuación generada por el usuario por el nuevo valor.
3. **Salir del programa:** Sale del bucle `while` ya que genera la condición `while == 3`.

RECURSOS UTILIZADOS:

El programa fue desarrollado usando bloques `if - elif - else` para evaluar situaciones y manejar la opción elegida por el usuario. Se usaron bloques `while` para que el programa se siga ejecutando hasta que el usuario desee salir.

En esta actividad se utilizó la Inteligencia Artificial Gemini para pulir la lógica que el programa debía seguir, para ayudar a corregir errores de ciertos parámetros ingresados por el usuario y para la comprensión e implementación de bibliotecas útiles de python.

Bibliotecas utilizadas:

math:

Es una librería estándar de python que tiene funciones matemáticas avanzadas ya listas para usar. En este caso, se implementó para calcular el ángulo de la función lineal. Su ventaja es la precisión y el ahorro de código y tiempo de crear cálculos desde cero cuando ya existen dentro de python. Solo sirve para cálculos individuales y no para procesar listas grandes de números a la vez.

numpy:

Esta librería es muy útil en el manejo de datos numéricos masivos. Se utilizó para generar de forma matemática el rango y una lista de 100 números continuos para el eje X. Su ventaja es la velocidad y el uso en conjunto con la librería matplotlib para generar gráficos.

matplotlib.pyplot:

El módulo `.pyplot` indica que se usa esa subcarpeta específica para dibujos simples de dos dimensiones dentro de la gran librería `matplotlib`.

Es una biblioteca muy usada para graficar funciones matemáticas. Agarra los puntos generados por la biblioteca `numpy`, los dibuja en la pantalla y los une con una línea para formar el gráfico final.

Abre una ventana que permite moverse por los ejes, hacer zoom o guardar el gráfico como una foto.

Se eligió esta porque era de las más sencillas de implementar en programas sencillos como el que requería.

Lógica de bloque de generación de gráficos:

Línea	Explicación
<code>x = np.linspace(-10, 10, 100)</code>	Se genera una lista con 100 números perfectamente distribuidos entre el -10 y el 10 para actuar como los valores del eje X.
Para recta: <code>y = m * x + b</code> Lineal Para parábola: <code>y = a * (x**2) + b</code> Cuadrática * x + c	Utilizando los coeficientes previamente ingresados por el usuario, Python calcula de manera simultánea el valor de Y para cada uno de esos 100 puntos utilizando las ecuaciones correspondientes a la función.
Para recta: <code>plt.plot(x, y, label="Función Lineal", color="red")</code> Para parábola: <code>plt.plot(x, y, label="Función Cuadrática", color="blue")</code> <code>plt.plot(vx, vy, 'ro', label=f"Vértice ({vx:.2f}, {vy:.2f}))"</code>	El comando <code>plt.plot(x, y)</code> funciona como un "conector de puntos": ubica las 100 coordenadas (x, y) invisibles en la pantalla y las une mediante segmentos rectos muy pequeños. Debido a la gran cantidad de puntos juntos, se percibe una curva o recta perfecta. El argumento color define el color del trazado de la línea. El argumento <code>label</code> le asigna una etiqueta de texto a cada gráfico. En el caso de la función cuadrática, se utilizó además la instrucción <code>plt.plot(vx, vy, 'ro')</code> para marcar específicamente el vértice como un punto rojo ('ro') destacado.
<code>plt.show()</code>	Renderiza y abre la ventana en el sistema operativo, pausando la terminal de comandos hasta que el usuario decida cerrar la imagen.

PARTE C - Análisis

Consigna 3 — Análisis de rendimiento en un sistema distribuido

Parte A — Comprensión del modelo

1. Interpretar y explicar con sus palabras:

- Qué representa cada fila de M

Cada fila de la matriz (M) representa una función o proceso del sistema distribuido. Una fila completa indica cuánto tarda esa misma función en cada servidor, lo que sirve para ver en cuál corre mejor.

- Qué representa cada columna de M

Cada columna representa un servidor distinto. Si se observan de arriba a abajo, muestran cuánto tardan las diferentes funciones en ejecutarse en un mismo servidor, determinando si funciona correctamente o si se está quedando lenta.

- Qué representa un valor cualquiera de la matriz

Cada valor de la matriz (M) representa el tiempo promedio de ejecución medido en milisegundos, de una función en un servidor específico. Por ejemplo, el número 120 que está en la primera fila y primera columna, significa que la Función 1 tarda un promedio de 120 milisegundos en ejecutarse dentro del Servidor 1.

2. Analizar las dimensiones de M y C y determinar si es posible calcular el producto $M*C$. Justificar matemáticamente.

Ambas matrices tienen 3 filas y 3 columnas. La dimensión de la primera (M) es de (3x3) y la de la segunda (C) es de (3x3). Por lo tanto, es posible calcular ($M*C$) ya que para multiplicar matrices, la cantidad de columnas de la primera debe ser igual a la cantidad de filas de la segunda. El resultado será una nueva matriz de dimensión (3x3).

Parte B — Implementación en Python

3. Calcular:

- El tiempo promedio de ejecución por función
- El tiempo promedio de ejecución por servidor

{PEGAR LINK DEL REPOSITORIO}

4. Calcular la matriz transpuesta de M y explicar qué representa en este contexto.

En la matriz original (M) las filas representan funciones y las columnas servidores. En la matriz transpuesta (MT) sucede al revés, las filas pasan a representar servidores mientras que las columnas pasan a representar funciones. Esto sirve para analizar de otra manera el rendimiento de cada servidor respecto a todas las funciones del sistema.

Parte C — Análisis mediante producto matricial

5. Calcular el producto $T=M*C$

```
T = | 9850  7150  7200 |  
    | 17500 10700 12200 |  
    | 8150  5500  5950 |
```


Link repositorio:

Link video: